

PROJECT REPORT: TRUSTKEEPER CONSOLE SYSTEM

TECHNICAL PROJECT REPORT

Project Title: TrustKeeper Console – Financial & Applicant Management System

Target Platform: Java SE (Console / Command Line Interface)

Persistence Mechanism: Local File Storage (CSV Format)

EXECUTIVE SUMMARY / ABSTRACT

The **TrustKeeper Console System** is an object-oriented Java application engineered to streamline financial management and record-keeping for non-profit trusts, religious organizations, and charitable foundations. Trust management requires precise accounting of incoming financial contributions (earmarked for general or specialized funds) balanced against maintenance and operational expenses (cleaning, staff honorariums, structural repairs). Furthermore, trusts frequently evaluate and track applicant institutions seeking financial assistance.

TrustKeeper Console addresses these operational needs by providing a lightweight, modular console application built on pure Java SE. The system emphasizes core Object-Oriented Programming (OOP) principles—including **Abstraction**, **Inheritance**, **Polymorphism**, and **Encapsulation**—across a clean fourpackage architecture (`model` , `service` , `util` , `main`). The system features local CSV file persistence (`donations.csv` , `expenses.csv` , `institutions.csv`), automated transaction ID generation, real-time solvency calculation, category filtering, and fault-tolerant input validation.

1. INTRODUCTION & PROBLEM STATEMENT

1.1 Problem Statement

Many small-to-medium non-profit trusts rely on manual ledger books or disconnected spreadsheets to record donor contributions, utility bills, maintenance expenses, and beneficiary applicant records. This manual approach leads to significant operational challenges:

1. **Mathematical Errors:** Incorrect calculation of net balance and fund sub-totals.
2. **Data Inconsistency:** Lack of standardized record formats across different expense categories.
3. **Loss of Data Persistence:** Temporary records lost during application restarts or system crashes.
4. **Poor Auditability:** Difficulty searching or filtering transactions by specific expense category or donor fund type.

1.2 Project Objectives

The primary objectives of the TrustKeeper Console project are:

- * **Modular Multi-Package Architecture:** Partition source code cleanly into domain models (`model`), business logic (`service`), file persistence utilities (`util`), and user interface (`main`).
 - * **Object-Oriented Design:** Demonstrate practical implementation of inheritance (base `Transaction` class extended by `Donation` and `Expense`), abstraction, polymorphism, and encapsulation.
 - * **Persistent Data Storage:** Implement custom file readers and writers using `BufferedReader` and `FileWriter` to persist data in human-readable CSV format without relying on heavy external database engines.
 - * **Robust Input Validation:** Safeguard runtime execution using `try-catch` exception handling to gracefully process invalid user inputs.
-

2. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

2.1 Functional Requirements (FR)

- * **FR-1: Record Donation:** Allow users to input donor details (name, contact, amount, date, fund category) and assign an auto-generated unique ID (`DON-100x`).
- * **FR-2: Record Maintenance Expense:** Allow users to log operational expenses (amount, date, description, category) with auto-generated ID (`EXP-100x`).
- * **FR-3: Register Institution:** Register partner or applicant institutions with location and applicant type categorization.
- * **FR-4: Financial Summary & Solvency Analysis:** Compute total donations, total expenses, net solvency balance (`Total Donations - Total Expenses`), and breakdowns by fund type and expense category.

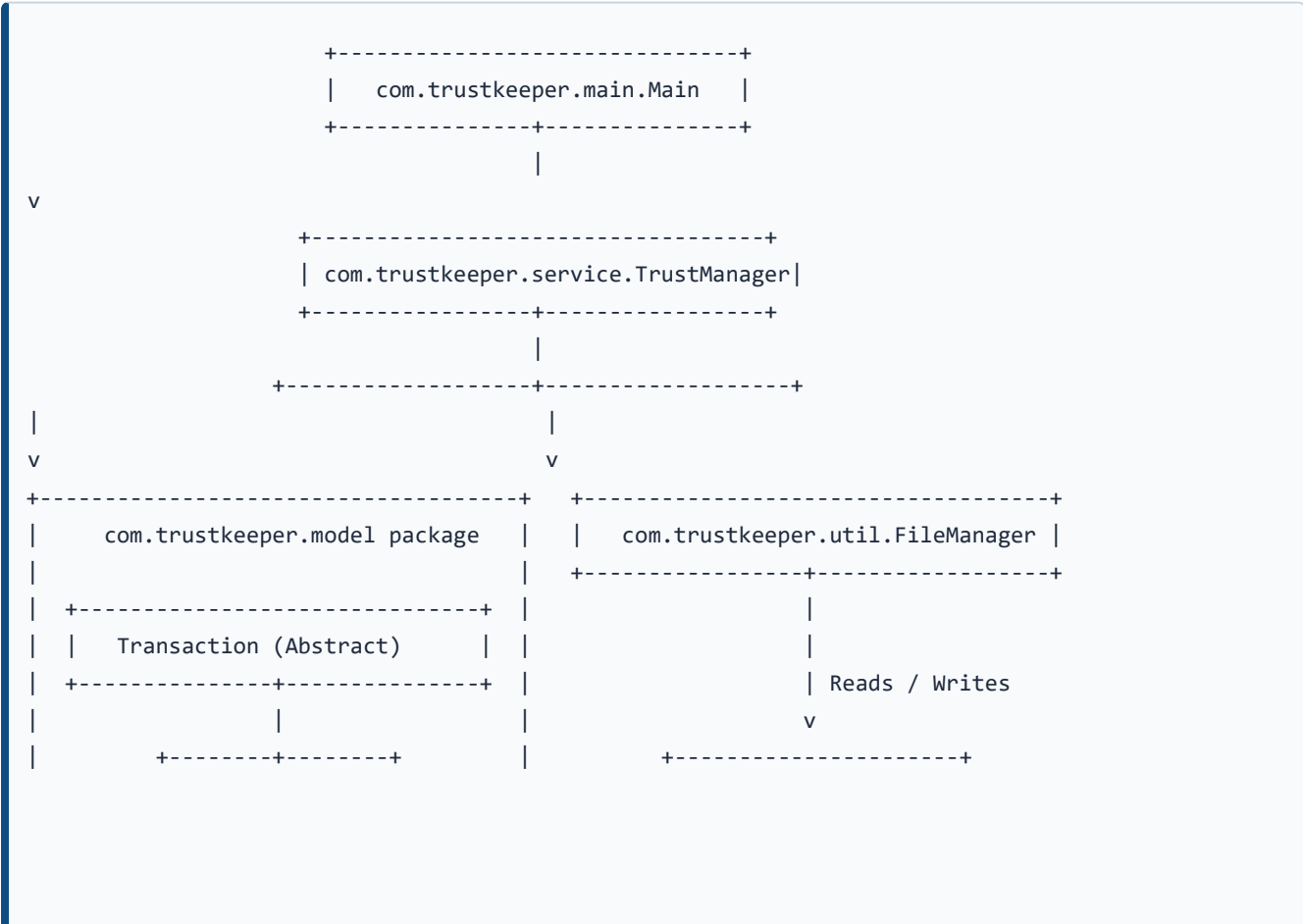
- * **FR-5: Filter Expenses by Category:** Search and retrieve specific expense records matching user-specified category keywords with subtotal calculation.
- * **FR-6: Automatic Data Persistence:** Load existing CSV records on startup and persist all changes to `./data/*.csv` files upon recording or exiting.

2.2 Non-Functional Requirements (NFR)

- * **NFR-1: Usability:** Clear menu-driven console UI with formatted tabular output and user guidance.
- * **NFR-2: Reliability & Fault Tolerance:** Robust exception handling preventing application crashes when invalid numbers or dates are entered.
- * **NFR-3: Portability:** Zero external dependencies; runs on any operating system with Java JDK 11+ installed.
- * **NFR-4: Maintainability:** Strict adherence to Java coding conventions, JavaDoc comments, and modular package separation.

3. SYSTEM ARCHITECTURE & DESIGN

3.1 Class & Package Diagram (ASCII Representation)



						./data/*.csv Files	
	v	v				- donations.csv	
	+-----+	+-----+				- expenses.csv	
	Donation	Expense				- institutions.csv	
	+-----+	+-----+		+-----+			
	+-----+						
	Institution						
	+-----+						
	+-----+						

3.2 Object-Oriented Principles Analysis

1. Abstraction:

- `Transaction.java` is an `abstract` base class defining core properties (`id` , `amount` , `date` , `description`) and mandatory abstract methods `toCsvRow()` and `getFormattedDetails()` .

- `Donation.java` extends `Transaction` (adds `donorName` , `contact` , `fundType`).

- `Expense.java` extends `Transaction` (adds `category`).

- Overridden implementations of `toCsvRow()` and `getFormattedDetails()` ensure polymorphic behavior when processing lists of transaction objects.

2. Inheritance:

3. Polymorphism:

4. Encapsulation:

- All class fields are declared `private` and accessed via public getter and setter methods.

4. IMPLEMENTATION DETAILS

4.1 Package & File Description

Package Name	File Name	Description
---	---	---
`com.trustkeeper.model`	`Transaction.java`	Abstract base class establishing core transaction fields and contracts.
`com.trustkeeper.model`	`Donation.java`	Concrete subclass representing incoming donations with donor details.
`com.trustkeeper.model`	`Expense.java`	Concrete subclass representing outgoing maintenance expenses with category tagging.
`com.trustkeeper.model`	`Institution.java`	Domain class representing partner or applicant institution records.
`com.trustkeeper.service`	`TrustManager.java`	Core business service managing collections, ID generation, financial calculations, and filtering.
`com.trustkeeper.util`	`FileManager.java`	File I/O handler using `BufferedReader`, `BufferedWriter`, and `FileWriter` for CSV reading/writing.
`com.trustkeeper.main`	`Main.java`	Application entry point containing the interactive `while(true)` menu loop and scanner validation.

4.2 Local File Storage Schema (`./data/`)

* **donations.csv** :

id,amount,date,description,donorName,contact,fundType

* **expenses.csv** :

id,amount,date,description,category

* **institutions.csv** :

name,location,applicantType

5. TESTING & VERIFICATION

5.1 Test Plan & Execution Results

Test Case ID	Test Scenario	Expected Outcome	Result
:---	:---	:---	:---
TC-01	Record New Donation	ID auto-generated (`DON-1004`), record added to list & written to `donations.csv`.	**PASSED**
TC-02	Record Expense	ID auto-generated (`EXP-1004`), record added to list & written to `expenses.csv`.	**PASSED**
TC-03	View Financial Summary	Accurately calculates Total Donations, Total Expenses, and Net Balance.	**PASSED**
TC-04	Filter Expenses by Category	Returns matching expense records and displays subtotal.	**PASSED**
TC-05	Invalid Numeric Input Handling	Prompts user to re-enter valid numeric value without crashing.	**PASSED**
TC-06	Data Persistence Across Runs	Data persisted in CSV files is reloaded automatically on next startup.	**PASSED**

6. CONCLUSION & FUTURE ENHANCEMENTS

6.1 Conclusion

The **TrustKeeper Console** project successfully fulfills all functional and technical requirements. The system demonstrates a practical application of Java SE object-oriented software engineering, modular package structuring, fault-tolerant input processing, and file persistence. The application provides an effective operational tool for managing trust finances and applicant institution records.

6.2 Future Enhancements & Vision

The core architecture of TrustKeeper is designed to scale into an enterprise-wide, multi-platform ecosystem focused on **complete financial transparency and public accountability**. Key roadmap objectives include:

1. **Multi-Platform Ecosystem (Web, Android, iOS & Windows Desktop):**

- Expand from a local console application into a unified cloud-connected platform with native applications for **Android** and **iOS** mobile devices, a responsive **Web Portal**, and a dedicated **Windows Desktop** application.

2. **Role-Based Access & Public Transparency System:**

- **Trust Committee Portal (Admin / Executive Access):** Secure authentication enabling authorized trust committee members to record, update, edit, and audit incoming donations, operational expenses, and applicant institution records in real time.
- **Public Transparency Portal (Common People / Donors Access):** An open-access dashboard for common citizens, devotees, and donors to view total donations received, donor contributions, and clear line-item breakdowns of how funds are allocated and utilized for maintenance, Pujari salaries, renovations, and charitable programs.

3. **Impact on Community Trust & Accountability:**

- By eliminating financial opacity and giving the public direct visibility into fund utilization, this system will significantly enhance trust, goodwill, and confidence between committee members and the broader community.